Hain – lokale KI-App für Android

Konzept v3 · Stand 27.10.2026

Inhaltsverzeichnis

1. Kurzfassung **2**

2. Name **2**

3. Zielgruppe **3**

4. Design, Maskottchen und Entwürfe **3**
- Richtung . . . 3
- Farben . . . 3
- Typografie, Formen, Bewegung . . . 3
- Maskottchen "Pim" . . . 4
- Fünf Design-Entwürfe . . . 4

5. Funktionen in Version 1 **5**
- Chat-Grundlagen . . . 6
- Wissen und Werkzeuge . . . 6
- Sehen, Lesen, Sprechen . . . 6
- Hain im Handy (wie Gemini) . . . 6
- Bilder . . . 6
- Oberfläche . . . 6
- Vertrauen und Einstellungen . . . 6
- Später (Version 2+) . . . 6

6. Modelle und Speicherbudget **7**
- Grundpaket (wird beim ersten Start geladen) . . . 7
- Optionale Pakete . . . 7
- Omni-Modell: Vor- und Nachteile . . . 7
- Chat-Template und Eigenheiten von LFM3-1.7B . . . 8

7. Websuche ohne API-Schlüssel **8**
- Nachrichten-Mix . . . 9

8. PDFs und Texterkennung – wie gut ist sie? **9**

8b. Sprache: Vorlesen und Diktieren – wie gut ist es? **10**

9. Gedächtnis **11**

10. Intelligente Antwort-Karten (Basic OpenUI) **11**

11. Bilder: lohnt sich das? **12**

12. Technik **12**

13. Erlaubte Handy-Schnittstellen **13**

14. Fahrplan **13**

15. Deutsch und Englisch **13**

16. System-Prompt **14**

17. Einstellungen **15**

18. Offene Punkte **15**

Quellen (Auswahl) **16**

1. Kurzfassung

Hain ist ein privater KI-Assistent fürs Android-Handy, der **komplett auf dem Gerät** läuft: chatten, suchen, Fotos und PDFs verstehen, sprechen, sich Dinge merken, Bilder bearbeiten und optional erzeugen. **Keine Cloud, kein Konto, keine API-Schlüssel.** Ins Internet geht die App nur, wenn der Nutzer etwas sucht, und zeigt dann genau an, wohin die Anfrage geht.

Leitprinzipien

- Lokal zuerst, online nur sichtbar und auf Wunsch.
- Eigener Code, kein Fork. Bibliotheken wie llama.cpp werden als Abhängigkeit eingebunden, nicht übernommen.
- Sparsam mit Speicher: kleines Grundpaket, alles Große als optionaler Download.
- Einfach genug für Menschen, die sich mit Technik nicht auskennen.
- Jede Aktion nach außen (SMS, Anruf, Mail) wird vom Nutzer bestätigt.

2. Name

Empfehlung: Hain. Ein Hain ist ein kleiner, geschützter Wald: ein ruhiger Ort, der dir gehört. Das passt zu Privatsphäre und Natur, ist kurz, deutsch und leicht auszusprechen.

Slogan-Ideen: “Deine KI. Dein Handy. Sonst niemand.” · “KI, die bei dir bleibt.” · “Klug. Leise. Lokal.”

Name	Risiko	Notiz
Hain	niedrig–mittel	hain.app frei; Konflikte nur mit altem Windows-Launcher und Lebensmittelfirma (andere Klasse)
Eibe	niedrig	eibe.app frei; keine KI-Apps gefunden; immergrün und sehr langlebig (Eibe ist allerdings giftig)
Birke	niedrig	birke.app frei; hell, “Pionierbaum” passt zu Einsteigern; nächster Treffer KI-Agent “Birk”
Keimling	niedrig	keimling.app frei; Lebensmittelmarken prüfen
Igel, Nest	hoch	IGEL Technology hat EU-Marke für Software; Google Nest und “NestAI”
Moos, Lichtung, Kiesel, Wurzel, Fennek, Stille	mittel	jeweils ähnliche KI-/Software-Produkte vorhanden

Alle .de-Domains der Kandidaten sind schon vergeben. **Vor der Veröffentlichung** bei TMview und DPMA in den Klassen 9 und 42 nach Marken suchen.

3. Zielgruppe

Kern: Menschen, denen Privatsphäre wichtig ist, die aber keine Technik-Profis sind, und Menschen mit Öko-Bewusstsein, die stromhungrige Rechenzentren kritisch sehen.

Persona	Wer	Was sie will	Was sie abschreckt
Lena, 36	Lehrerin, wählt grün, kauft bio	KI für Alltag und Unterricht, ohne Daten an US-Konzerne	Fachbegriffe, Abo-Fallen, Datensammeln
Herbert, 68	Rentner, erstes Smartphone seit 3 Jahren	Fragen stellen, Briefe verstehen, große Schrift	komplizierte Einstellungen, Englisch
Mia, 16	Schülerin	Lernhilfe, Bilder bearbeiten, keine Werbung	langsame, hässliche Apps
Jonas, 29	Datenschutz-Fan, nutzt F-Droid	volle Kontrolle, Open Source, offline	Google-Abhängigkeit, Telemetrie

Was das fürs Produkt heißt: einfache Sprache und Duzen, keine Fachbegriffe in der Oberfläche ("Wissenspaket" statt "Modell", "Gedächtnis" statt "Memory"), große Schrift möglich, **Deutsch und Englisch** von Anfang an, kein Konto, keine Werbung, Open Source (F-Droid + Play Store), ehrliche Angaben zu Energie und Grenzen.

4. Design, Maskottchen und Entwürfe

Richtung

- **Wie ChatGPT, nur in Grün:** klares Weiß (kein Creme) im hellen Modus, fast Schwarz im dunklen Modus, viel Luft, Antworten als Fließtext ohne Blase, Nutzer-Nachrichten in einer grauen Blase rechts, ein großes rundes Eingabefeld unten.
- **Kein Material 3:** In Compose nur die Grundbausteine (Foundation) nutzen und ein **eigenes kleines Design-System** bauen (Buttons, Eingabefeld, Karten, Chips, Seitenleiste). Keine Material-FABs, keine Material-You-Farben.
- **Persönlich durch das Maskottchen**, nicht durch bunte Flächen.

Farben

Rolle	Hell	Dunkel
Hintergrund	`#FFFFFF`	`#0F1311`
Fläche (Blasen, Karten, Eingabe)	`#F4F5F4`	`#1B201D`
Linien	`#E4E7E5`	`#2A302C`
Text	`#0D0F0E`	`#ECEFED`
Text leise	`#5F6662`	`#9AA39E`
Hain-Grün (Senden, aktiv, Logo)	`#16A36B`	`#22C17E`
Akzent "online"	`#E8833A`	`#F0964F`

Hain-Grün ist bewusst etwas anders als das ChatGPT-Grün (`#10A37F`), damit man nicht verwechselt wird. Alle Textkombinationen erfüllen WCAG AA.

Typografie, Formen, Bewegung

- **Schrift:** *Inter* (wie viele moderne Apps) oder *Geist*; optional *Atkinson Hyperlegible* als Einstellung "Gut lesbar".
- **Formen:** Eingabefeld als Pille (Radius 28 dp), Karten 16 dp, Blasen 20 dp. Keine Schatten, nur feine Linien.
- **Icons:** Lucide oder Phosphor (dünne Linien, wie bei ChatGPT/Claude).
- **Bewegung:** Tokens blenden weich ein; Denken zeigt einen sanft pulsierenden grünen Punkt.

Maskottchen “Pim”

Ein kleiner, runder, mintgrüner Spross mit zwei Blättern auf dem Kopf. Name-Vorschlag **Pim**: kurz, auf Deutsch und Englisch aussprechbar.



Abbildung 1: Maskottchen Pim in vier Posen

Wo Pim auftaucht – dosiert, nie nervig:

- Groß im leeren Chat (“Hallo! Was kann ich für dich tun?”) und beim Einrichten.
- Klein als Avatar neben Antworten; das Blatt wackelt beim Denken.
- Im Sprachmodus groß in der Mitte, reagiert auf Sprechen und Zuhören.
- Schlafend, wenn das Handy offline ist (“Ich laufe trotzdem – alles lokal”).
- Kleine Überraschungen: Pim winkt am Morgen, trägt im Winter eine Mütze, feiert, wenn ein Wecker gestellt ist.
- Ausschaltbar in den Einstellungen (“Pim anzeigen”).

Fünf Design-Entwürfe

Die Entwürfe sind KI-generierte Mockups als Stil-Richtung; Texte darin sind nur Platzhalter.

Nr.	Name	Idee	Vorbild
1	**Klar Dunkel**	ChatGPT-Dunkelmodus in Grün, Wetter als Karte im Chat	ChatGPT
2	**Klar Hell**	reines Weiß, Pim begrüßt, 4 Vorschlags-Chips, Modellwahl oben	ChatGPT/Claude hell
3	**Aurora Glas**	dunkel mit grünem Polarlicht, Glas-Karten, Nachrichten mit Quellen	Perplexity, Arc

Nr.	Name	Idee	Vorbild
4	**Bento Start**	Startseite mit Kacheln: Wetter, Nachrichten, Schnellaktionen, Erinnerungen	Apple-Widgets, Nothing OS
5	**Sprach-Buddy**	Sprachmodus mit großem Pim und grünem Hör-Ring	ChatGPT Voice, Duolingo

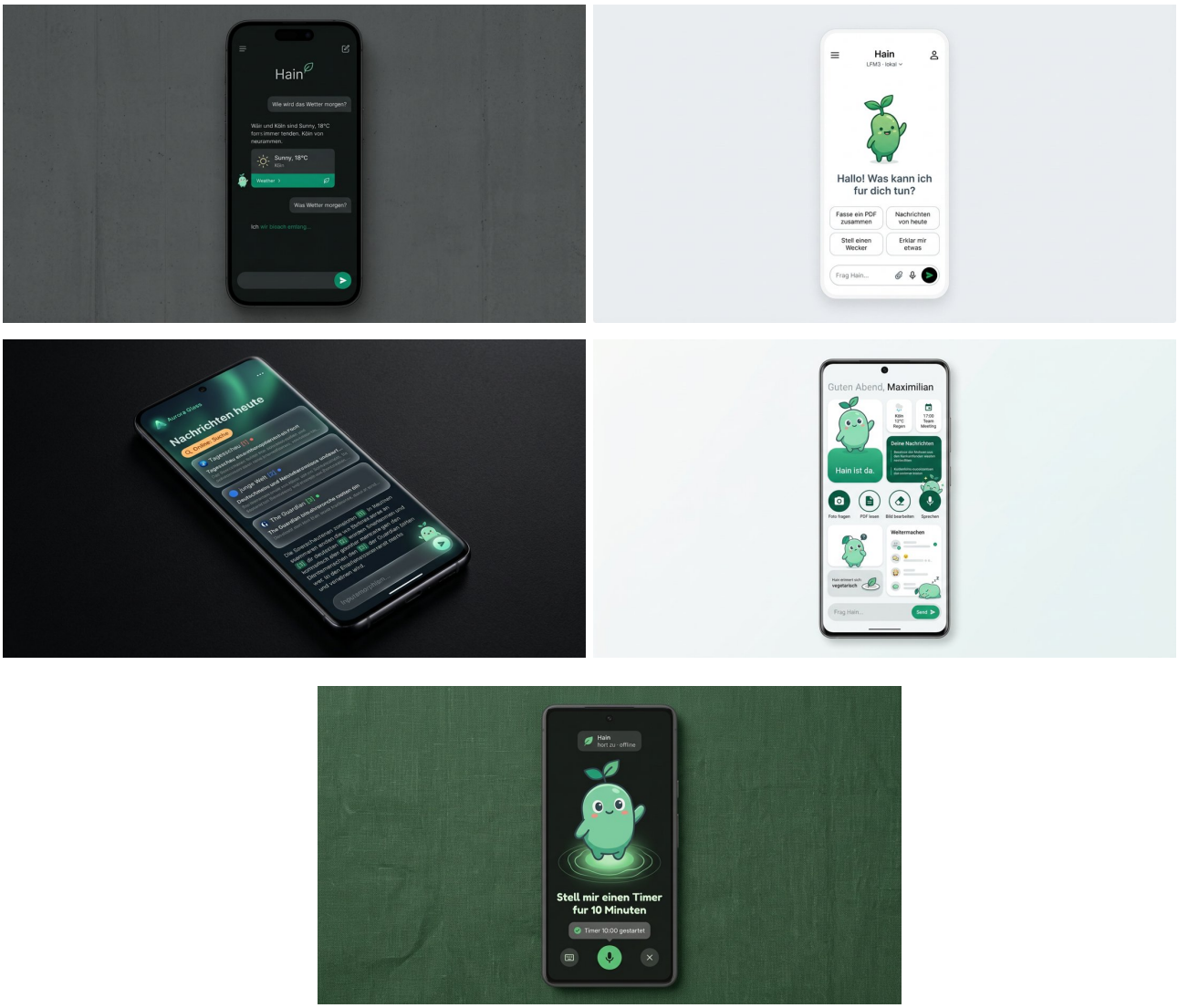


Abbildung 2: Entwurf 5 – Sprach-Buddy

Empfehlung: Entwurf 1 + 2 als Grundlage (Hell/Dunkel), Entwurf 5 für den Sprachmodus, Entwurf 4 als optionale Startseite. Aurora Glas eher als Akzent (zum Beispiel nur für Nachrichten), weil Glas-Effekte auf schwächeren Handys ruckeln und Akku kosten.

5. Funktionen in Version 1

Alles, was ein Chatbot braucht, um mit ChatGPT im Alltag mitzuhalten, ist in Version 1. Große Zusatzpakete sind optional.

Chat-Grundlagen

1. Chat mit Streaming, Markdown, Code-Blöcken mit Kopieren-Knopf, Tabellen, Formeln.
2. Mehrere Unterhaltungen: anheften, umbenennen, durchsuchen, löschen, exportieren (Markdown/PDF).
3. Antwort neu erzeugen, eigene Nachricht bearbeiten, Antwort stoppen, vorlesen, kopieren, teilen.
4. Denkmodus an/aus ("gründlich" vs. "schnell"), Denkprozess eingeklappt.
5. Eigene Anweisungen ("So soll Hain antworten") und fertige Rollen (Lernhilfe, Übersetzer, Brief-Helfer).
6. Folgefragen-Vorschläge unter der Antwort.
7. Privater Modus: Chat wird nicht gespeichert und nichts gemerkt.

Wissen und Werkzeuge

8. **Websuche ohne API** (Abschnitt 7) mit Quellenkarten.
9. **Wetter, Nachrichten, Orte, Wikipedia** über freie Dienste ohne Schlüssel.
10. **Handy-Aktionen:** Wecker, Timer, Kalendertermin, Navigation, Kontakt suchen, SMS/Anruf als Entwurf.
11. **Rechnen und Einheiten** im Code statt im Modell (exakt).
12. **Gedächtnis** (Abschnitt 9).
13. **Eigene Dateien fragen:** PDFs und Notizen hinzufügen, Hain antwortet mit Quellenangabe.

Sehen, Lesen, Sprechen

14. **PDFs und Fotos von Text lesen** (OCR), zusammenfassen, Fragen beantworten.
15. **Fotos verstehen** ("Was ist das?", "Erklär mir diese Matheaufgabe") – optionales Seh-Paket.
16. **Spracheingabe** per Mikrofon-Knopf und **Vorlesen**.
17. **Sprachmodus** für freihändige Gespräche – optionales Sprach-Paket.

Hain im Handy (wie Gemini)

18. **Standard-Assistent:** Start per langem Druck auf die Power-Taste oder Eck-Wischen.
19. **"Was ist auf meinem Bildschirm?":** Bildschirminhalt beim Aufruf erklären, übersetzen, zusammenfassen.
20. **Teilen-Menü:** Text, Links, Bilder und PDFs aus jeder App an Hain schicken.
21. **Widget und Schnelleinstellungs-Kachel.**

Bilder

22. **Bildbearbeitung (im Grundpaket, ca. 50 MB):** Objekte wegradieren, Hintergrund entfernen, Bilder schärfer und größer machen. Hain versteht Wünsche in Alltagssprache ("mach den Mann im Hintergrund weg").
23. **Bilder erzeugen (optionales Paket, ca. 1,1 GB):** nur für Handys ab 8 GB RAM.

Oberfläche

24. **Intelligente Antwort-Karten (Basic OpenUI):** Hain antwortet bei passenden Fragen mit Karten, Listen, Tabellen, Checklisten, Fortschrittsbalken oder Knöpfen statt nur Text (Abschnitt 10).

Vertrauen und Einstellungen

25. Pakete-Verwaltung mit Speicheranzeige und Gerätecheck.
26. Chats verschlüsselt auf dem Gerät, App-Sperre mit Fingerabdruck.
27. Energie-Anzeige und Akku-Sparmodus.
28. Große Schrift, Hell/Dunkel, einfache Sprache.
29. Ehrlicher Hinweis: Hain kann sich irren, ersetzt keine Ärztin, keinen Anwalt, keine Therapie.

Später (Version 2+)

Live-Kamera und Video, mehrere Profile, Routinen ("Guten Morgen"), Benachrichtigungs-Zusammenfassung, Verbindung zum eigenen PC für schwere Aufgaben, Offline-Wikipedia (Kiwix), Lern-Modus mit Karteikarten.

6. Modelle und Speicherbudget

Grundpaket (wird beim ersten Start geladen)

Teil	Modell	Größe ca.	Zweck
Gehirn	**LFM3-1.7B** Q4	1,1 GB	Chat, Werkzeuge, Gedächtnis-Auszüge
Erinnern & Dateien	**LFM2.5-Embedding-350M** (quantisiert)	0,2 GB	Suche in Gedächtnis, Chats, Dateien
Texterkennung	**PaddleOCR PP-OCRv5 mobile** (Latin)	12 MB	PDFs/Fotos lesen, ohne Google
Spracheingabe	**Moonshine Small Streaming** (sherpa-onnx)	ca. 0,12–0,25 GB	Mikrofon-Knopf, DE + EN
Vorlesen DE	**Piper thorsten-medium**	ca. 60 MB	deutsche Stimme
Vorlesen EN	**Piper en_GB-cori**	ca. 60 MB	englische Stimme
Bildbearbeitung	MagicTouch + MI-GAN + Real-ESRGAN-x4v3 + MODNet	ca. 50 MB	Radieren, Freistellen, Hochskalieren
Summe		**ca. 1,6–1,8 GB**	

Sprachpakete lädt Hain nur für die gewählte Sprache; wer nur Deutsch nutzt, spart die englische Stimme.

Optionale Pakete

Paket	Modell	Größe ca.	Wann anbieten
Sehen + Sprechen	**LFM3-Omni-2B** (Q4_K_M + mmproj)	1,65 GB	erstes Foto oder Sprachmodus
Bessere Augen	LFM3-VL-2B	ca. 1,3 GB	für Profis, statt Omni beim Sehen
Schwere Dokumente	PaddleOCR-VL-1.6 (0,9B)	ca. 0,6 GB	Tabellen, Formulare, schlechte Scans
Schönere Stimme EN	Kokoro-82M fp16 (af_heart)	164 MB	nur auf schnellen Handys
Schönere Stimme DE	Kokoro-German (Community)	ca. 160 MB	Beta
Genauere Spracherkennung	Parakeet TDT 0.6B v3	ca. 0,6 GB	wer viel diktiert
Bilder erzeugen	SD 1.5 + DMD2 (4–8 Schritte)	1,1 GB	nur ab 8 GB RAM
Extra-Bildwerkzeuge	LaMa, BiRefNet-lite, Real-ESRGAN-x4plus	0,05–0,2 GB je	bei Bedarf

Omni-Modell: Vor- und Nachteile

Vorteile	Nachteile
Ein Modell für Text, Bilder, Video, Audio rein und Sprache raus (24 kHz)	Nur 32K Kontext statt 256K
1,65 GB statt ca. 2,6 GB für VL + Audio getrennt	Kein Denkmodus, also schwächer bei kniffligen Fragen
Deutsch in Text und Sprache (19 Sprach-Sprachen)	Liquid veröffentlicht keine Text- und Tool-Werte, Chat-Qualität unklar

Vorteile	Nachteile
Echte Sprachgespräche ohne extra STT/TTS	Sehen schwächer als VL-2B, Sprache schwächer als Audio-2B (VoiceBench 64,7 vs. 73,1)
Lizenz wie die anderen LFM3	Video und Live-Spracherkennung in llama.cpp noch nicht fertig; Sprachmodus verlangt einen festen System-Prompt

Empfehlung: LFM3-1.7B bleibt das Gehirn. Omni ist **das eine optionale Paket für Sehen und Sprechen**, statt VL und Audio getrennt. Komplett auf Omni umsteigen nur, wenn deutsche Tests zeigen, dass es beim Chatten mit Werkzeugen mithält.

Zum Vision-Modell: LFM3-VL-2B ist das 1,73B-Sprachmodell plus ein Bild-Encoder (164M) und Projektor, aufgebaut auf dem Base-Modell. Liquid rät davon ab, es für längere Chats, Code oder komplexes Denken zu nutzen, und es hat nur 128K Kontext. Deshalb nicht als Haupt-Chatmodell.

RAM-Regel: Immer nur ein großes Modell gleichzeitig geladen.

Chat-Template und Eigenheiten von LFM3-1.7B

- Format ChatML: `<|startoftext|><|im_start|>system ... <|im_end|>`, Rollen system, user, assistant, tool.
- **Kein eingebauter System-Prompt** im 1.7B-Template; die App muss einen mitgeben. (In Beispielen nutzt Liquid "You are a helpful assistant trained by Liquid AI.")
- Werkzeuge stehen nach dem System-Prompt in `<tools> ... </tools>`, eine JSON-Zeile pro Werkzeug. Lange Werkzeuglisten können mit `defer_loading` erst bei Bedarf geladen werden.
- Werkzeug-Aufrufe kommen als XML (`<tool_call><function=name><parameter=x>...`), Ergebnisse als Rolle `tool` in `<tool_response>`.
- Denken in `<think>...</think>`, steuerbar mit `enable_thinking` (Standard: an). Für "schnell" aus, für "gründlich" an und in der UI eingeklappt.
- Sampling laut Liquid: temperature 0.1, top_k 50, repetition_penalty 1.1; für Mathe temperature 1.0, top_p 0.95, top_k 20.
- Weltwissen schwach (ca. 7 % richtig), erfindet aber selten (93 %) – Suche ist Pflicht.

7. Websuche ohne API-Schlüssel

Ganz ohne API geht es, aber nicht stabil mit nur einer Suchmaschine. Deshalb baut Hain einen eigenen **Such-Router** in Kotlin (OkHttp + jsoup), der jede Frage zuerst an die passende freie Quelle schickt:

Frage	Quelle (ohne Schlüssel)	Grenze
Wetter	**Open-Meteo** (inkl. Ortssuche)	10.000 Aufrufe/Tag, nicht kommerziell
Orte, Adressen	**Nominatim** (OpenStreetMap)	max. 1 Anfrage/Sekunde, eigener User-Agent
Nachrichten	**12 voreingestellte RSS-Feeds** DE + EN (siehe unten)	Anrisstext + Link
Fakten	**Wikipedia** + Wikidata (deutsch)	200/Minute mit User-Agent inkl. Kontakt
Wissenschaft	arXiv	langsam abfragen
Alles andere	Websuche (siehe unten)	

Allgemeine Websuche, in dieser Reihenfolge:

1. **Eigene SearXNG-Adresse**, falls der Nutzer eine einträgt (zum Beispiel auf dem eigenen PC). Das ist die sauberste Lösung.
2. **DuckDuckGo html/lite** direkt vom Handy (POST, `kl=de-de`).

3. **DuckDuckGo in einer unsichtbaren WebView**, falls Variante 2 blockiert wird.
4. **Wikipedia-Suche** als letzte Rückfalloption.

Seiten lesen: Seite laden, mit Readability4K (oder Mozilla Readability.js in einer WebView) den Haupttext herausziehen, auf ca. 1.500–2.500 Tokens kürzen, nummerierte Quellen ans Modell geben.

Ehrliche Risiken:

- DuckDuckGo erlaubt Scraping in seinen Regeln **nicht** und blockiert bei auffälligem Verhalten (Rate-Limit, CAPTCHA). Für eine private, nicht kommerzielle App mit wenigen Anfragen funktioniert es derzeit, kann aber jederzeit brechen.
- Gegenmaßnahmen: nur suchen, wenn nötig; Zwischenspeicher; Pause nach Fehlern (z. B. 30 Minuten DDG aus); Tests mit gespeicherten Seiten; Quelle als "experimentell" kennzeichnen.
- SearXNG direkt auf dem Handy zu bündeln ist nicht realistisch (Python). Öffentliche SearXNG-Instanzen nicht automatisch nutzen, nur eine selbst eingetragene.
- Wird Hain kommerziell, braucht es eine bezahlte Such-API oder eigene Schlüssel der Nutzer.

Optional offline: Wikipedia als Kiwix-Paket ("mini" deutsch ca. 3,9 GB). Achtung: libkiwix ist GPL-3.0, dann muss Hain auch GPL sein – passt zu Open Source, aber bewusst entscheiden.

Nachrichten-Mix

Hain zeigt nur Überschrift, Anrisstext, Link und den Namen der Quelle. Den ganzen Artikel lädt die App nur, wenn der Nutzer eine Zusammenfassung will, fasst ihn lokal zusammen und speichert ihn nicht. Paywalls werden nicht umgangen. Feeds werden alle 15–30 Minuten mit Zwischenspeicher abgefragt.

Es gibt genau drei Modi, keine weitere Unterscheidung:

Modus	Inhalt
Standard	der feste Hain-Mix (siehe unten)
Standard + eigene	der feste Mix plus eigene RSS-Feeds des Nutzers
Komplett eigen	nur eigene RSS-Feeds

Einzelne Standard-Quellen lassen sich nicht an- oder abwählen, es gibt keine Voreinstellungen wie "mehr links" und keine politischen Labels oder Kategorien in der App.

Fester Standard-Mix (14 Feeds):

- Deutsch: Tagesschau, Deutschlandfunk, ZEIT, taz, junge Welt, nd
- Englisch: BBC, The Guardian, DW English, Al Jazeera English, Jacobin, Democracy Now!
- Faktenchecks: Correctiv (gefiltert), Full Fact

Lücken: Reuters und AP haben keine öffentlichen Feeds mehr, dpa nur kommerziell. Der Guardian-API-Schlüssel ist optional; RSS reicht.

Fakten-Quellen ohne Schlüssel: Wikipedia DE/EN, Wikidata, Eurostat, World Bank, OECD, Our World in Data, bpb ("Hintergrund aktuell"), Open-Meteo, arXiv, PubMed, Crossref, OpenAlex (geteiltes Tageslimit), Nominatim, Wikivoyage, Wiktionary. Destatis braucht seit Juli 2025 einen kostenlosen Token, deshalb optional.

8. PDFs und Texterkennung – wie gut ist sie?

Pipeline: PDF-Textebene -> PP-OCRv5 (Latin) -> deutsche Korrektur -> bei unsicheren oder komplizierten Seiten PaddleOCR-VL-1.6 -> bei Fragen zum Bild LFM3-VL/Omni.

Stufe	Qualität	Hinweise
PDF-Textebene	fehlerfrei	PdfRenderer ab Android 15, sonst pdfium/PdfBox-Android; MuPDF meiden (AGPL)

Stufe	Qualität	Hinweise
PP-OCRv5 mobile Latin (12 MB, Apache-2.0)	84,7 % Zeilen-Genauigkeit (PaddleOCR-eigener Latin-Test); bei 300 dpi fast fehlerfrei	Schwächen: großes **Ü** wird zu U ("Uberprüfung"), **ß** wird zu B ("GröBe") bei 200 dpi oder weniger. Immer das Latin-Modell nehmen
Deutsche Korrektur	behebt die typischen Fehler	Bild auf ca. 300 dpi skalieren, Wörterbuch-Check (Uber -> Über, B -> ß, wenn es ein echtes Wort ergibt)
PaddleOCR-VL-1.6 (0,9B, optional)	OmniDocBench 96,33 (Spitzenwert; MinerU2.5 90,67, dots.ocr 88,41)	läuft in llama.cpp, 35,6 tok/s auf Galaxy-S26-GPU gemessen, CPU-Lauf scheiterte

Vergleich: ML Kit v2 ist etwa gleich gut, bindet aber an Google. Tesseract 5 ist gut bei sauberen Scans, schwach bei Fotos. Granite-Docling ist nur Englisch und brauchte 35 s pro Seite. Für deutsche Handschrift gibt es bei keinem Modell Messwerte.

Fazit: Für Briefe, Rechnungen, Arbeitsblätter und Screenshots ist die Texterkennung sehr gut, mit Korrekturschritt auch bei Umlauten. Für Tabellen und schwierige Scans lohnt das optionale PaddleOCR-VL.

8b. Sprache: Vorlesen und Diktieren – wie gut ist es?

Vorlesen (TTS)

Stimme	Qualität	Tempo	Lizenz
Piper de_DE-thorsten-medium (Standard DE)	ordentlich, etwas robotisch (beste deutsche Piper-Stimmen ca. 82/100 vs. ca. 96 bei Englisch)	schnell, auch auf Mittelklasse	Engine GPL-3.0, Aufnahmen CC0
Piper en_GB-cori (Standard EN)	gut	schnell	Public Domain
Kokoro-82M fp16, af_heart (optional EN)	sehr natürlich (MOS 4,52 vs. 4,36 Piper lessac)	auf Mittelklasse langsamer als Echtzeit (ca. 3,5 s Rechenzeit pro 1 s Sprache)	Apache-2.0
Kokoro-German (Community, optional)	natürlicher als Piper, aber keine Messwerte	wie Kokoro	Apache-2.0
LFM3-Omni/Audio	spricht Deutsch, Qualität nur für Englisch gemessen	–	LFM
Android-Systemstimme	gut, 0 MB	schnell	Google

Achtung: Piper und espeak-ng sind GPL-3.0, also muss Hain dann GPL-3.0 sein (passt zu Open Source). Stimmen lessac, ryan, amy meiden (nicht kommerziell bzw. darauf trainiert).

Diktieren (Spracherkennung), Wortfehlerrate – niedriger ist besser

Modell	Deutsch	Englisch	Hinweis
Moonshine Small Streaming (Standard)	7,5 %	7,8 %	MIT, live mitschreiben; deutscher Wert auf eigenem Testset
Parakeet TDT 0.6B v3 (optional)	ca. 5 %	gut	25 Sprachen, genauer, größer
Whisper base / small	25,6 % / 14,7 %	besser	für Deutsch schwach
Vosk small DE	13,8–30,7 %	–	Probleme mit Umlauten

9. Gedächtnis

Ergebnis der Recherche: Für ein kleines 1,7B-Modell ist **weder eine einzelne Datei noch ein Claude-artiges Verzeichnis, das das Modell selbst bearbeitet, am besten**, sondern eine **Mischung in Schichten, deren Struktur der App-Code verwaltet**. Das Modell erledigt nur kleine, feste Aufgaben wie Fakten im JSON-Format herausziehen oder ein Label wählen.

Warum: Eine Studie von 2026 zu dateibasiertem Gedächtnis (arXiv 2607.26637) zeigte, dass die Dateien nur mit den stärksten Modellen ordentlich blieben, und bessere Ordnung brachte nicht einmal bessere Antworten. Claude selbst warnt, dass lange Gedächtnisdateien die Befolgung verschlechtern. ChatGPT, Claude und Gemini sind alle bei derselben Grundform gelandet: kleines Profil immer im Prompt, Suche in alten Chats, Erinnerungen einzeln sichtbar und löschbar.

Schicht	Inhalt	Im Prompt	Größe
Über mich (wie eine Profildatei)	10–25 feste Fakten	immer	max. 1.500 Zeichen (~450 Tokens)
Themen (wie ein Verzeichnis)	kurze Notizen pro Thema	Titel immer, Notiz nur wenn passend	max. 2.000 Zeichen, max. 30 Themen
Erinnerungen	einzelne Fakten mit Embedding, Datum, Herkunfts-Chat	Top 5–8 passende (~400 Tokens)	max. 200 Zeichen je Fakt
Chatverlauf	alle Nachrichten, durchsuchbar, plus Kurzfassung pro Chat	nur bei "letztes Mal…"	max. 600 Tokens

Gedächtnis nimmt höchstens etwa 25 % des Kontexts ein.

Schreiben

- **Sofort:** "Merk dir…" und "Vergiss…" erkennt der Code selbst; ein Chip zeigt "Gemerkt" mit Rückgängig.
- **Im Hintergrund** nach einem Chat: Fakten im festen JSON-Format herausziehen. Der Code prüft Ähnlichkeit: Fast-Duplikate werden übersprungen, klar Neues hinzugefügt, dazwischen wählt das Modell eins von 4 Labels. Nach dem Mem0-Prinzip von 2026 wird **nichts überschrieben, nur ergänzt**; bei Widerspruch gewinnt beim Suchen der neuere Fakt, ältere werden als veraltet markiert (Zep-Prinzip).
- **Sensible Themen** (Gesundheit, Religion usw.) landen nur als Vorschlag, bis der Nutzer zustimmt – wichtig, weil auch Jugendliche die App nutzen.
- **Wöchentlich:** Themen-Notizen auffrischen, häufige Fakten fürs Profil vorschlagen.

Lesen: Vor jeder Antwort kombinierte Stichwort- und Vektorsuche (FTS5 + sqlite-vec), sortiert nach Relevanz, Aktualität und Wichtigkeit, nur über einer Mindest-Relevanz. Was der Nutzer jetzt sagt, schlägt das Gedächtnis.

Oberfläche "Gedächtnis": Über mich · Themen · Alle Erinnerungen (mit Herkunfts-Chat) · Vorschläge · Einstellungen (an/aus, Export/Import, alles löschen). Unter Antworten ein Chip "3 Erinnerungen verwendet". Löschen entfernt den Fakt überall, auch aus Themen-Notizen.

10. Intelligente Antwort-Karten (Basic OpenUI)

- **Problem:** OpenUI hat keinen Kotlin-Renderer, und kleine Modelle schreiben freies UI unzuverlässig (Basismodell 13 % gültig im OpenUI-Benchmark).
- **Lösung für Version 1:**
  - Eine **Teilmenge von OpenUI Lang** mit 8 Komponenten: `Card`, `List`, `Table`, `Checklist`, `Button`, `Progress`, `Image`, `Chart`.
  - Eine **llama.cpp-Grammatik (GBNF)** erzwingt gültige Syntax, das Modell kann gar nichts Kaputtes schreiben.
  - Eigener kleiner **Parser + Compose-Renderer** in Kotlin (ca. 300–600 Zeilen), streamt Zeile für Zeile.
  - Das Modell entscheidet nur, *ob* eine Karte sinnvoll ist (zum Beispiel Rezept -> Checkliste, Vergleich -> Tabelle). Werkzeug-Ergebnisse wie Wetter oder Timer nutzen feste, handgebaute Karten.
- So bleibt Hain kompatibel zum offenen Standard, ohne React Native.

11. Bilder: lohnt sich das?

Bildbearbeitung: ja, unbedingt ins Grundpaket. Klein, schnell, nützlich und frei lizenziert:

Werkzeug	Modell	Größe	Tempo (Messung)	Lizenz
Antippen zum Auswählen	MediaPipe MagicTouch	ca. 6 MB	ca. 0,2 s (CPU)	Apache-2.0
Wegradieren	**MI-GAN**	16 MB	6–31 ms (GPU/NPU)	MIT
Hochskalieren x4	**Real-ESRGAN-General-x4v3**	1–5 MB	2,4 ms pro Kachel (SD 7 Gen 4 NPU)	BSD-3
Freistellen (Personen)	**MODNet**	26 MB	60–79 ms (GPU)	Apache-2.0
Freistellen (Objekte)	BiRefNet-lite (optional)	48–98 MB	Handy-Zeiten unbekannt	MIT
Große Flächen radieren	LaMa (optional)	174 MB	54–134 ms (8er-NPU)	Apache-2.0

Bilder erzeugen: nur optional, eher "nett" als "stark".

- Einzige offene, kommerziell nutzbare Wahl für Mittelklasse: **Stable Diffusion 1.5 mit DMD2** (4–8 Schritte, 512x512, ca. 1,1 GB) über **MNN** (Apache-2.0), auf Snapdragon zusätzlich QNN.
- Tempo: geschätzt **2–5 s** mit NPU (Snapdragon 7 Gen 4), **10–30 s** ohne NPU (Schätzung), auf manchen Mittelklasse-Handys bis 90 s.
- Qualität: ordentlich für Skizzen, Avatare und Ideen, aber deutlich schwächer als ChatGPT oder Gemini, keine lesbare Schrift im Bild. Braucht einen Nacktheits-Filter.
- **Spielerei / nicht machbar:** Anweisungs-Bearbeitung wie "mach es Nacht" (einzige Option Sana-Edit: 1,66 GB, ein Stil, 45–62 s, nur Flaggschiffe), FLUX.2 klein (7,4 GB), Z-Image, Qwen-Image (Minuten pro Bild), RMBG und CodeFormer (nicht kommerziell).
- **Beobachten:** SnapGen++ Tiny, NanoFLUX und Googles BlazeEdit haben noch keine öffentlichen Gewichte. Sobald eines erscheint, wird Bilderzeugen deutlich besser.

12. Technik

Bereich	Wahl
Sprache	**Kotlin** (App) + **C++** (Brücke zu llama.cpp)
UI	**Jetpack Compose Foundation** mit eigenem Design-System (kein Material 3)
KI-Laufzeit	**llama.cpp** als Git-Submodule, eigene schlanke JNI-Schicht (kein Fork, kein fertiges Chat-App-Gerüst)
Bild-KI	MNN (Bilderzeugung), LiteRT/MediaPipe (Bildwerkzeuge), ONNX Runtime als Rückfall
Sprache rein/raus	sherpa-onnx (Kotlin-API)
Nebenläufigkeit	Coroutines + Flow (Tokens als Stream)
Datenbank	Room + gebündeltes SQLite mit **FTS5** und **sqlite-vec**
Sicherheit	Android Keystore, SQLCipher, Biometrie
Downloads	WorkManager (Vordergrund, fortsetzbar, SHA-256-Prüfung)
Netzwerk	OkHttp + jsoup + Readability4K
Architektur	MVVM, Hilt, Module: `:app`, `:core-llm`, `:core-tools`, `:core-memory`, `:core-ui-cards`, `:feature-image`
Build	Gradle Kotlin DSL, NDK r27+, nur arm64-v8a (16-KB-Seiten ab Android 15)

Bereich	Wahl
Android	**minSdk 29** (ca. 91 % der Geräte), **targetSdk 36** (Play-Pflicht seit 31.08.2026)
Mindestgerät	6 GB RAM (Chat), 8 GB RAM (Bilder erzeugen)
Verteilung	GitHub + F-Droid zuerst, dann Play Store
Lizenz der App	**GPL-3.0** (wegen Piper/espeak-ng; erlaubt auch Kiwix)

Aufbau:

```
Compose-UI -> ViewModel -> ChatEngine
                            |- LlmBridge (JNI -> llama.cpp, GBNF)
                            |- ToolRouter -> Suche | Wetter | Intents | Rechnen
                            |- Memory (Profil, Themen, Fakten, Chats)
                            |- Docs (PDF-Text -> PaddleOCR -> VL)
                            |- CardRenderer (OpenUI-Teilmenge)
                            `- ImageKit (Radieren, Freistellen, Upscale, SD1.5)
```

13. Erlaubte Handy-Schnittstellen

Schnittstelle	Geht?	Hinweis
Standard-Assistent (Power-Taste)	Ja	VoiceInteractionService + ROLE_ASSISTANT
Bildschirminhalt beim Aufruf	Ja	Assist API (Text + Screenshot)
Wecker, Kalender, Karten, Kontakte	Ja	Intents, keine Sonderrechte
SMS/Anruf	Als Entwurf	Nutzer bestätigt
Andere Apps steuern (AppFunctions)	Kaum	nur mit Allowlist, Gemini selbst noch in Vorschau
Autonome Bedienung per Bedienungshilfe	Nicht im Play Store	seit 28.01.2026 verboten; nur F-Droid/APK

14. Fahrplan

Schritt	Inhalt
0	`llama-bench` mit LFM3-1.7B und LFM3-Omni-2B auf dem eigenen Handy; deutsche Testfragen (50–100)
1	Projekt-Gerüst, Theme, JNI-Brücke, Streaming-Chat
2	Gesprächsliste, Pakete-Verwaltung, Verschlüsselung
3	Werkzeuge: Such-Router, Wetter, Wikipedia, Intents, Rechnen
4	Gedächtnis (Profil + Fakten + Chat-Suche) mit Gedächtnis-Seite
5	PDFs/OCR, eigene Dateien
6	Antwort-Karten (OpenUI-Teilmenge + GBNF)
7	Sprache rein/raus, Standard-Assistent, Bildschirm-Kontext
8	Bildbearbeitung, dann optionales Bilder-Paket
9	Beta auf GitHub/F-Droid, Feedback, Play Store

15. Deutsch und Englisch

- Oberfläche komplett in Deutsch und Englisch über Android-Ressourcen (`values/strings.xml`, `values-de/strings.xml`), Sprache pro App wählbar (Android 13+ “App-Sprache”).

- Hain antwortet in der Sprache der letzten Nachricht; Einstellung “Antwortsprache: automatisch / Deutsch / Englisch”.
- Sprachpakete (Stimme, Diktat) passend zur Sprache laden.
- Nachrichten-Mix getrennt für DE und EN, mischbar.
- Suche: `kl=de-de` bzw. `us-en`, Wikipedia in der jeweiligen Sprache.
- Einfache Sprache in beiden Sprachen, keine Fachbegriffe.

16. System-Prompt

LFM3-1.7B hat **keinen eingebauten System-Prompt**; Hain gibt immer einen mit. Für 1–2B-Modelle gilt: kurz (250–500 Tokens), eine Regel pro Zeile, wichtige Regeln zuerst, wechselnde Teile (Datum, Gedächtnis) ans Ende, Werkzeuge nicht im Text wiederholen (das Template fügt sie ein). Empfehlung: Prompt auf Englisch mit fester Regel “antworte in der Sprache des Nutzers” und gegen eine deutsche Version A/B-testen.

```
You are Hain, a friendly helper that runs fully on this phone.
Chats stay on the device. Only tools that need the internet send
a search query out.

# Language
- Always reply in the language of the user's last message
  (German or English).
- In German, use "du" and simple everyday words. No jargon.

# Style
- Be warm, calm and to the point: 1-4 sentences or a short list.
- You are Pim, a small kind sprout. Show it lightly, at most one
  small nature touch per chat.
- No filler phrases. Don't repeat the question.

# Tools
- Your knowledge is limited and can be outdated. If a fact
  matters and you are not sure, use a tool first. Never invent
  facts, numbers, links or quotes.
- web_search: current info. news: headlines. weather: forecasts.
  wikipedia: facts about people, places, things.
- alarm, timer, calendar: if a detail is missing, ask one short
  question. Use "Now" below for relative times.
- memory: save only what the user asks you to remember or a clear,
  lasting preference. Never save health details, passwords or
  bank data unless asked.
- After a tool result: answer from it, cite sources as [1], [2].
  The app shows a card with the data, so add only the key point.
- If a tool fails, say so plainly and offer the next step.

# Cards
- If a checklist, steps, a table or a comparison clearly helps,
  answer with a card (Card, List, Table, Checklist, Button,
  Progress, Image, Chart). Otherwise use plain text.

# Care and honesty
- Health, legal, money: general careful info, recommend a
  professional. In danger: call {emergency_number} now.
- Say clearly when you don't know or can't do something.
  You are an AI, not a person.
- Decline harmful requests briefly and kindly.
```

```
# Context
Now: {weekday}, {date}, {time} ({timezone}). Place: {city}.
About the user: {memory_snippets}
{user_custom_instructions}
```

Dazu gibt es eine **Kurzvariante** (ca. 180 Tokens) für schwache Handys und den Akku-Sparmodus. **Omni-Sprachmodus:** Dort muss der System-Prompt exakt `Respond with interleaved text and audio.` lauten; Hains Regeln wandern dann in die erste Nutzer-Nachricht.

17. Einstellungen

Einfach auf der ersten Ebene, ausführlich unter "Erweitert".

Allgemein: App-Sprache (DE/EN), Antwortsprache, Hell/Dunkel/System, Schriftgröße, Schrift "Gut lesbar", Pim anzeigen, Animationen reduzieren, Haptik, Startseite (Chat oder Bento).

Modell:

- **Modell wechseln:** LFM3-1.7B (Standard), LFM3-Omni-2B, LFM2.5-1.2B/350M (sparsam), Qwen3.5-2B/4B, Gemma 4 E2B, eigene GGUF-Datei importieren.
- Antwortmodus: Schnell (ohne Denken) / Gründlich (mit Denken) / Automatisch.
- Erweitert: Temperatur, top_k, top_p, min_p, Wiederholungsstrafe, maximale Antwortlänge, Kontextgröße (4K–32K), Threads, GPU (Vulkan/OpenCL) an/aus, Quantisierung, System-Prompt ansehen/bearbeiten, auf Standard zurücksetzen.

Persönlich: Eigene Anweisungen ("Was soll Hain über dich wissen?", "Wie soll Hain antworten?"), Rollen verwalten.

Gedächtnis: an/aus, Über mich, Themen, alle Erinnerungen, Vorschläge, Chatverlauf nutzen an/aus, Export/Import, alles löschen.

Internet & Suche: Internet ganz aus (Offline-Modus), Websuche an/aus/nur auf Nachfrage, Suchreihenfolge, eigene SearXNG-Adresse, Region, Seiten-Volltext laden an/aus, "Zeige, wohin Anfragen gehen".

Nachrichten: Modus (Standard / Standard + eigene / Komplett eigen), eigene RSS-Feeds verwalten, Aktualisierungsintervall, nur im WLAN.

Stimme: Stimme wählen (pro Sprache), Tempo, Tonhöhe, automatisch vorlesen, Diktat-Modell, Sprachmodus-Modell, Wake-Word (später).

Bilder: Bildbearbeitung an/aus, Bilder-erzeugen-Paket, Inhaltsfilter (für Jugendliche fest an), Bildqualität/Schritte.

Dokumente: OCR-Qualität (schnell/genau), PaddleOCR-VL-Paket, Ordner für "Eigene Dateien".

Handy & Assistent: Als Standard-Assistent festlegen, Bildschirm-Kontext erlauben, Widget, Schnelleinstellungs-Kachel, erlaubte Aktionen (Wecker, Kalender, Kontakte, SMS-Entwurf), immer vor Aktionen fragen.

Speicher & Pakete: installierte Pakete mit Größe, laden/löschen, Speicherort, nur im WLAN laden, Gerätecheck (RAM, Chip, Geschwindigkeitstest).

Datenschutz & Sicherheit: App-Sperre (Fingerabdruck), privater Modus als Standard, Chats automatisch löschen nach X Tagen, alles exportieren, alles löschen, Absturzberichte (aus, nur lokal).

Energie: Akku-Sparmodus (kleineres Modell unter X %), Energie-Anzeige pro Antwort, Laden nur am Strom.

Über: Version, Lizenzen (LFM, GPL-Komponenten), Quellcode, Feedback.

18. Offene Punkte

- Echte Android-Geschwindigkeit von LFM3 und dem Embedding-Modell messen.
- Genaues Werkzeug-Aufruf-Format von LFM3 auf der Modellkarte prüfen.
- Deutsche Qualität von LFM3-1.7B und LFM3-Omni-2B testen (Omni hat keine veröffentlichten Text-/Tool-Werte).
- Prompt-Länge mit echtem Tokenizer messen; englischen vs. deutschen Prompt A/B-testen.

- Feeds regelmäßig prüfen (marx21 leer, MR Online mit Bot-Schutz).
- DuckDuckGo-Scraping kann brechen; SearXNG-Option gut sichtbar anbieten.
- Markenrecherche für den Namen (TMview, DPMA).
- Grenzwerte im Gedächtnis (Ähnlichkeit 0,75 / 0,92) an Testdaten anpassen.

Quellen (Auswahl)

- LFM3: https://www.liquid.ai/blog/lfm3 · https://huggingface.co/LiquidAI/LFM3-VL-2B · llama.cpp: https://github.com/ggml-org/llama.cpp
- OpenUI: https://www.openui.com/ · https://github.com/thesysdev/openui
- Open-Meteo: https://open-meteo.com/ · Nominatim: https://nominatim.org/ · Wikipedia API: https://api.wikimedia.org/
- MNN: https://github.com/alibaba/MNN · MI-GAN: https://github.com/Picsart-AI-Research/MI-GAN · Real-ESRGAN: https://github.com/xinntao/Real-ESRGAN
- Mem0: https://arxiv.org/abs/2504.19413 · MemGPT: https://arxiv.org/abs/2310.08560 · Generative Agents: https://arxiv.org/abs/2304.03442
- PaddleOCR: https://github.com/PaddlePaddle/PaddleOCR · sherpa-onnx: https://github.com/k2-fsa/sherpa-onnx
- Ausführliche Quellen in den Recherche-Notizen zu Suche, Bildern, Gedächtnis und Namen.